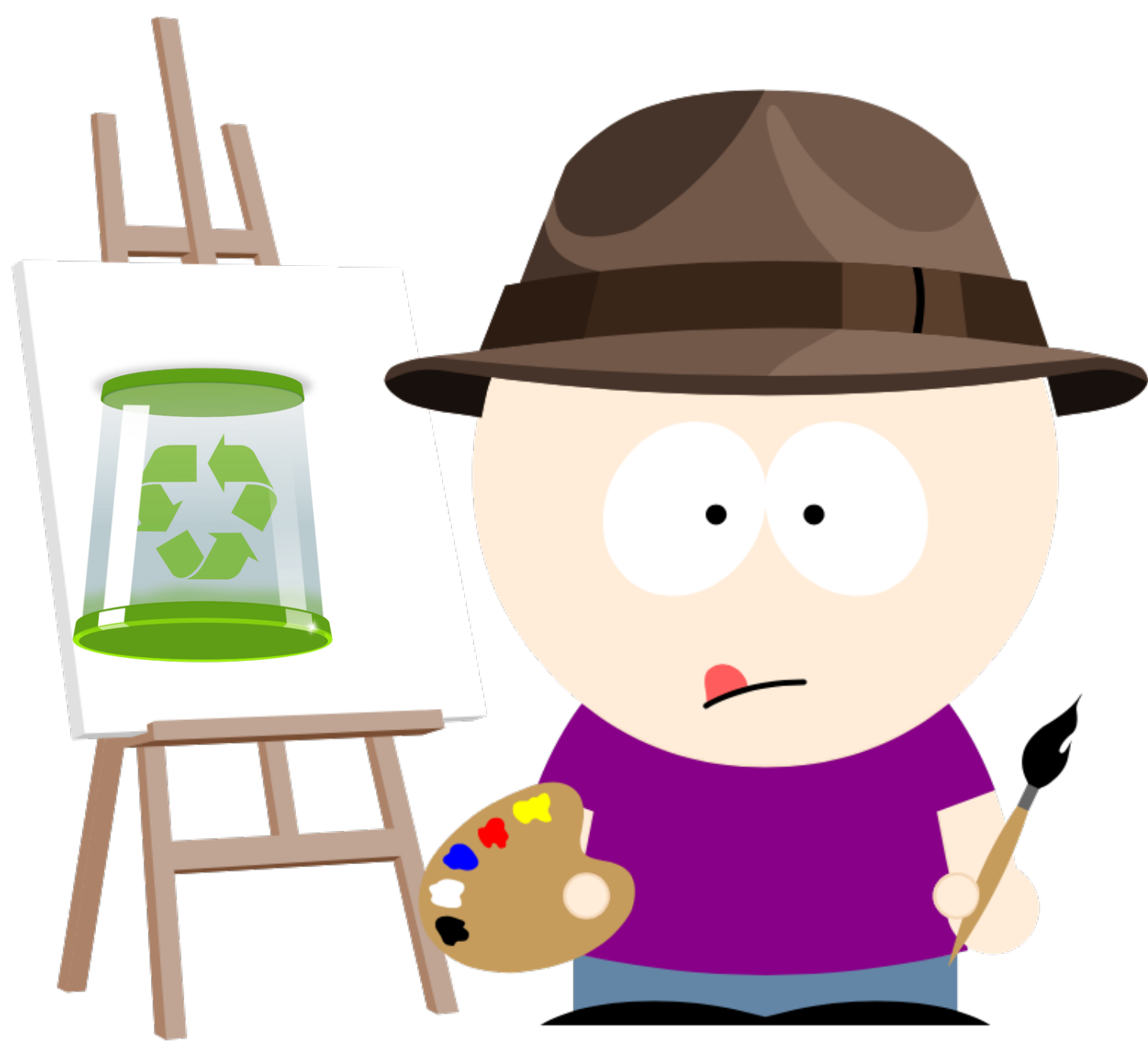




IN-CLASS EXERCISES



GARBAGE COLLECTION 2

Download exercises

1. Go to

<https://ligerlabs.org/compilers.html>

2. Download the file

`gc-2-exercises.zip`

3. Open up a terminal and Unzip the file

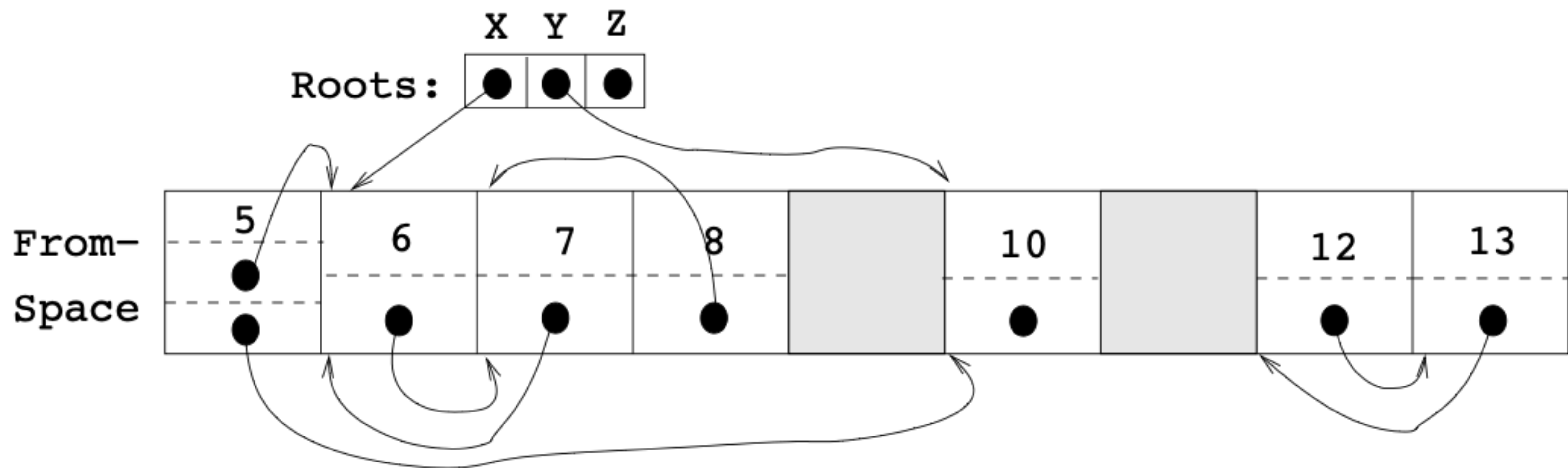
```
> unzip gc-2-exercises.zip
```

```
> cd gc-2-exercises
```

```
> ls
```

Task 1

- The heap in the figure on the next slide holds 7 objects. All objects have one integer field and one or two pointer fields (black dots).
- The only roots are the three global variables X, Y, and Z. Free space is shaded.
- Show the state of To-Space after a copying garbage collection has been performed on From-Space. Note that several answers are possible, depending on the visit strategy (Depth-First or Breadth-First Search) you chose.



Task 2

- Briefly describe these 4 GC algorithms:

Mark-and-Sweep

Mark-and-Compact

Copying
Collection

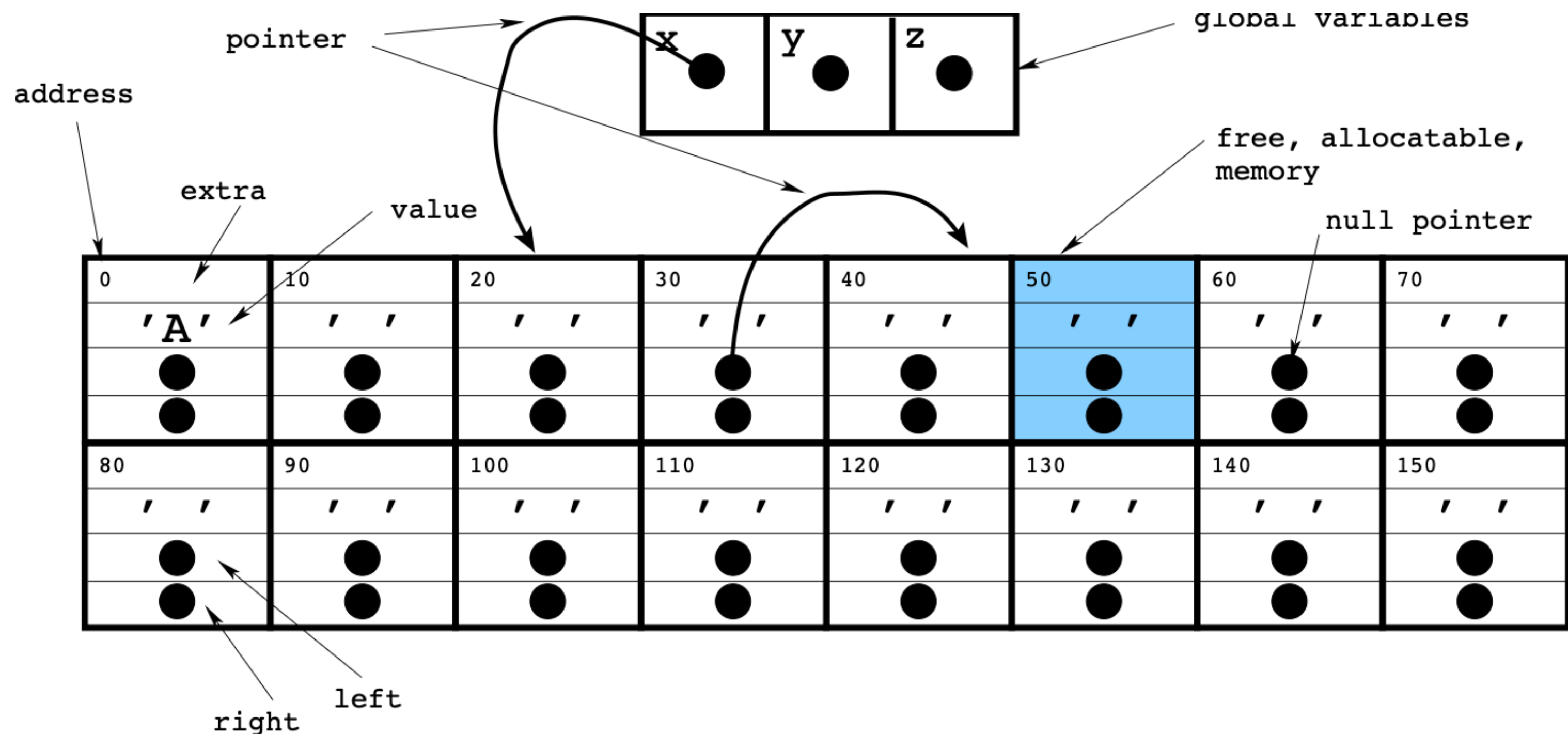
Generational
Collection

Task 3

- Describe the Deutsch-Schorr-Waite algorithm!
- When is it used?
- Why is it used?
- How does it work?

Task 4(a)

- Consider the Java program Cons on the next page. This program allocates memory on a heap of size 160 bytes, where each Cons node takes up 10 bytes, which we illustrate like this:



- Here, Extra refers to any additional information that a garbage collection algorithm might want to store in each node.

```
class Cons {
    public Cons left;
    public Cons right;
    public char value;

    public Cons(
        char value,
        Cons left,
        Cons right) {
        this.value = value;
        this.left = left;
        this.right = right;
    }

    static Cons x, y, z;
```

```
public static void main(String[] args) {
    x = new Cons('A', null, null);
    Cons b = new Cons('B', x, null);
    x.left = b;

    Cons f = new Cons('F', null, null);
    Cons e = new Cons('E', f, null);
    Cons d = new Cons('D', null, e);
    z = new Cons('C', d, e );

    b = d = f = e = null;

    <<BEFORE>>

    x = null;
    z.right.left = null;

    <<AFTER>>

    }
}
```


1. What does the heap look like at the point <<**BEFORE**>>, assuming that no **mark-and-compact** GC has occurred? Assume that nodes are allocated on the heap in the order that the new:s occur in the program.



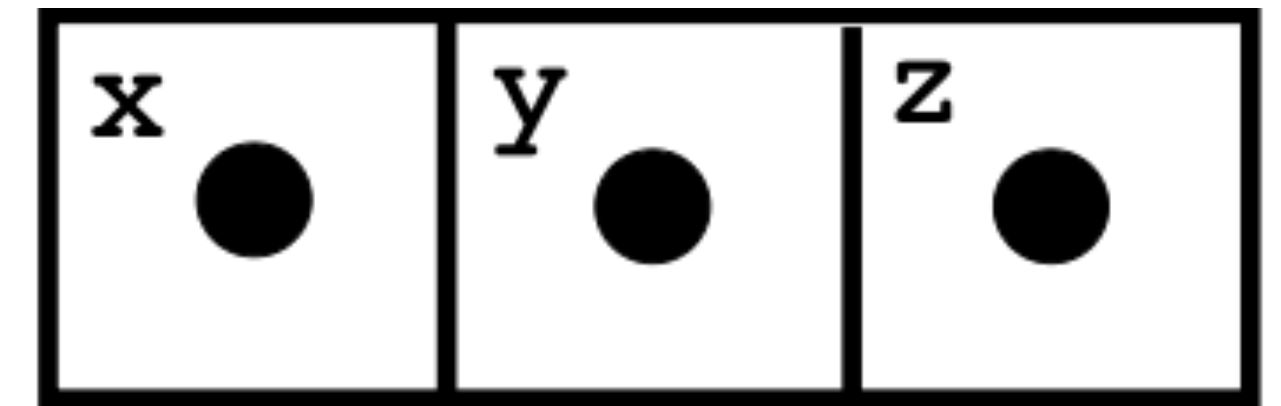
0	10	20	30	40	50	60	70
/	/	/	/	/	/	/	/
●	●	●	●	●	●	●	●
●	●	●	●	●	●	●	●
80	90	100	110	120	130	140	150
/	/	/	/	/	/	/	/
●	●	●	●	●	●	●	●
●	●	●	●	●	●	●	●

2. What does the heap look like at the point <<AFTER>>, assuming that a **mark-and-compact** GC occurred?
Mark all memory that the program is now able to allocate by shading the free nodes.



0	10	20	30	40	50	60	70
/	/	/	/	/	/	/	/
●	●	●	●	●	●	●	●
●	●	●	●	●	●	●	●
80	90	100	110	120	130	140	150
/	/	/	/	/	/	/	/
●	●	●	●	●	●	●	●
●	●	●	●	●	●	●	●

3. What does the heap look like at the point <<BEFORE>>, assuming that no **copying collection** GC has occurred? Assume that nodes are allocated on the heap in the order that the new:s occur in the program.



0	10	20	30	40	50	60	70
/	/	/	/	/	/	/	/
●	●	●	●	●	●	●	●
●	●	●	●	●	●	●	●
80	90	100	110	120	130	140	150
/	/	/	/	/	/	/	/
●	●	●	●	●	●	●	●
●	●	●	●	●	●	●	●

4. What does the heap look like at the point <<AFTER>>, assuming that a **copying collection** GC occurred? Mark all memory that the program is now able to allocate by shading the free nodes.

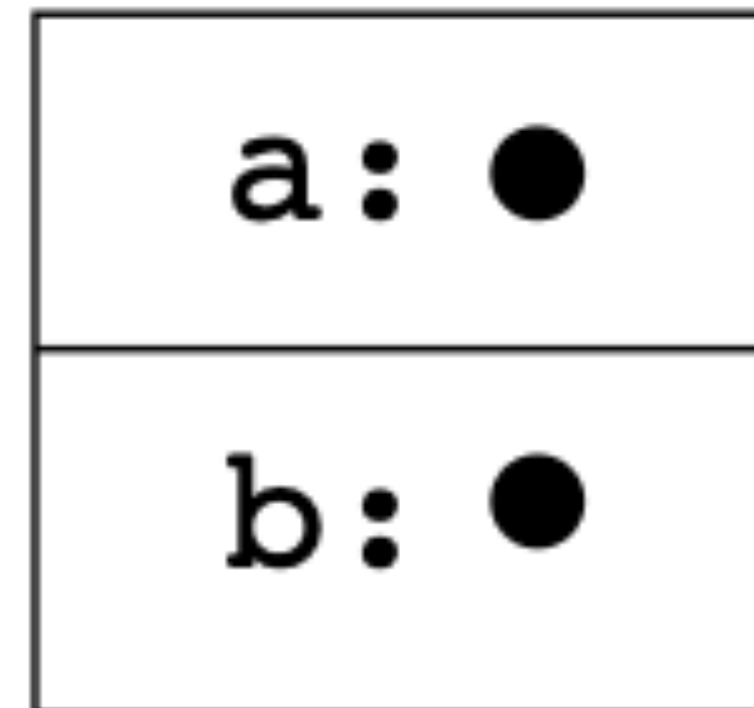


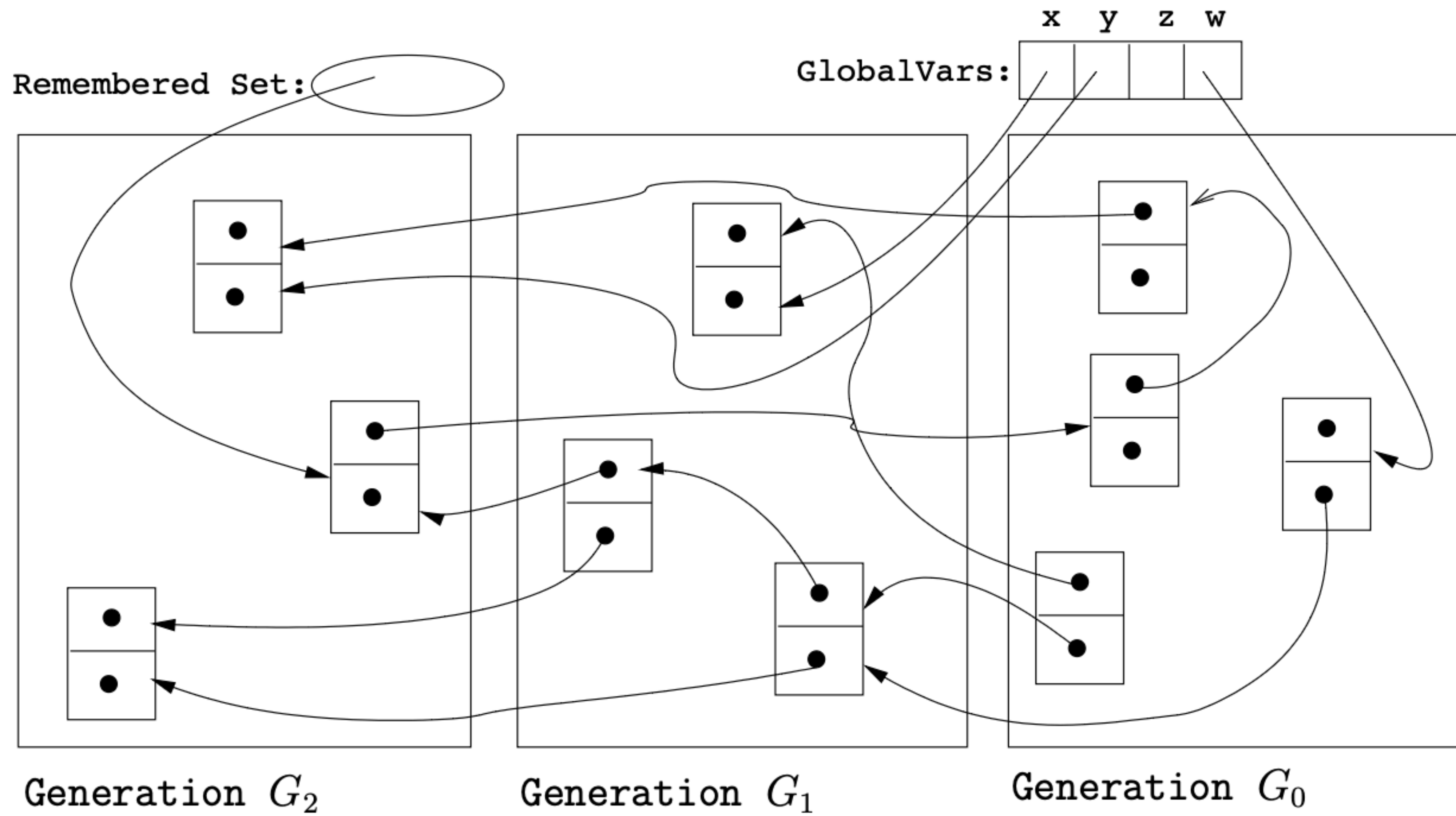
0	10	20	30	40	50	60	70
/	/	/	/	/	/	/	/
●	●	●	●	●	●	●	●
●	●	●	●	●	●	●	●
80	90	100	110	120	130	140	150
/	/	/	/	/	/	/	/
●	●	●	●	●	●	●	●
●	●	●	●	●	●	●	●

Task 5

- At time t_0 program P has constructed a heap that looks like on the next page.
- All the objects created by P are of the same type T, which has two pointer fields a and b:

TYPE T =





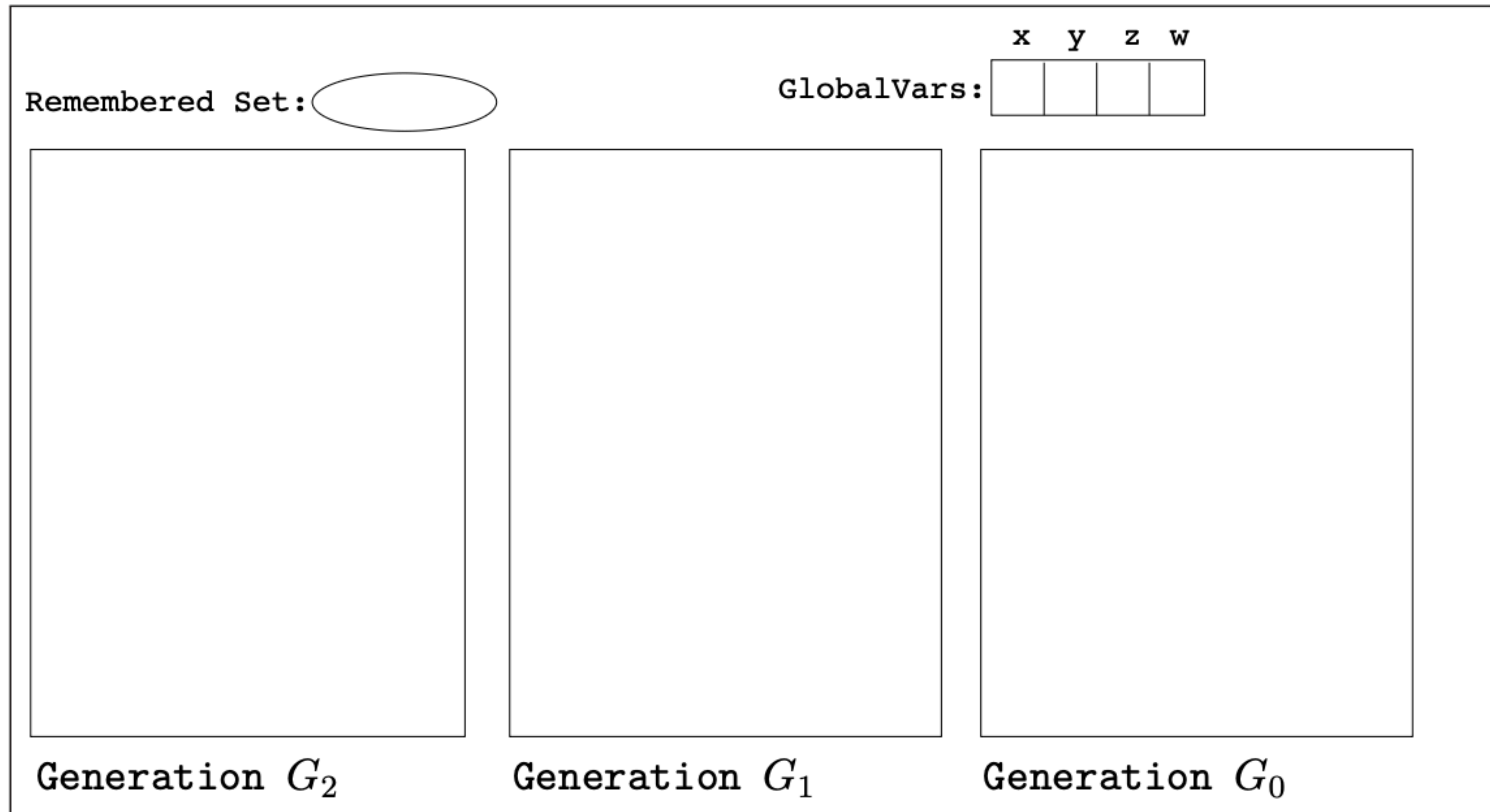
Task 5 (c)

- Then, at times $t_1 \dots t_9$, P performs the following actions:
- Here, $GC(G_0)$ means to perform a generational garbage collection of the youngest generation G_0 .
- $GC(G_0, G_1)$ means to perform a collection of the two youngest generations G_0 and G_1 .

```
t1 : w := nil;  
t2 : GC(G0);  
t3 : z := new T;  
t4 : z.a := x;  
t5 : z.b := null;  
t6 : y.b := z;  
t7 : x := null;  
t8 : z := null;  
t9 : GC(G0, G1);
```

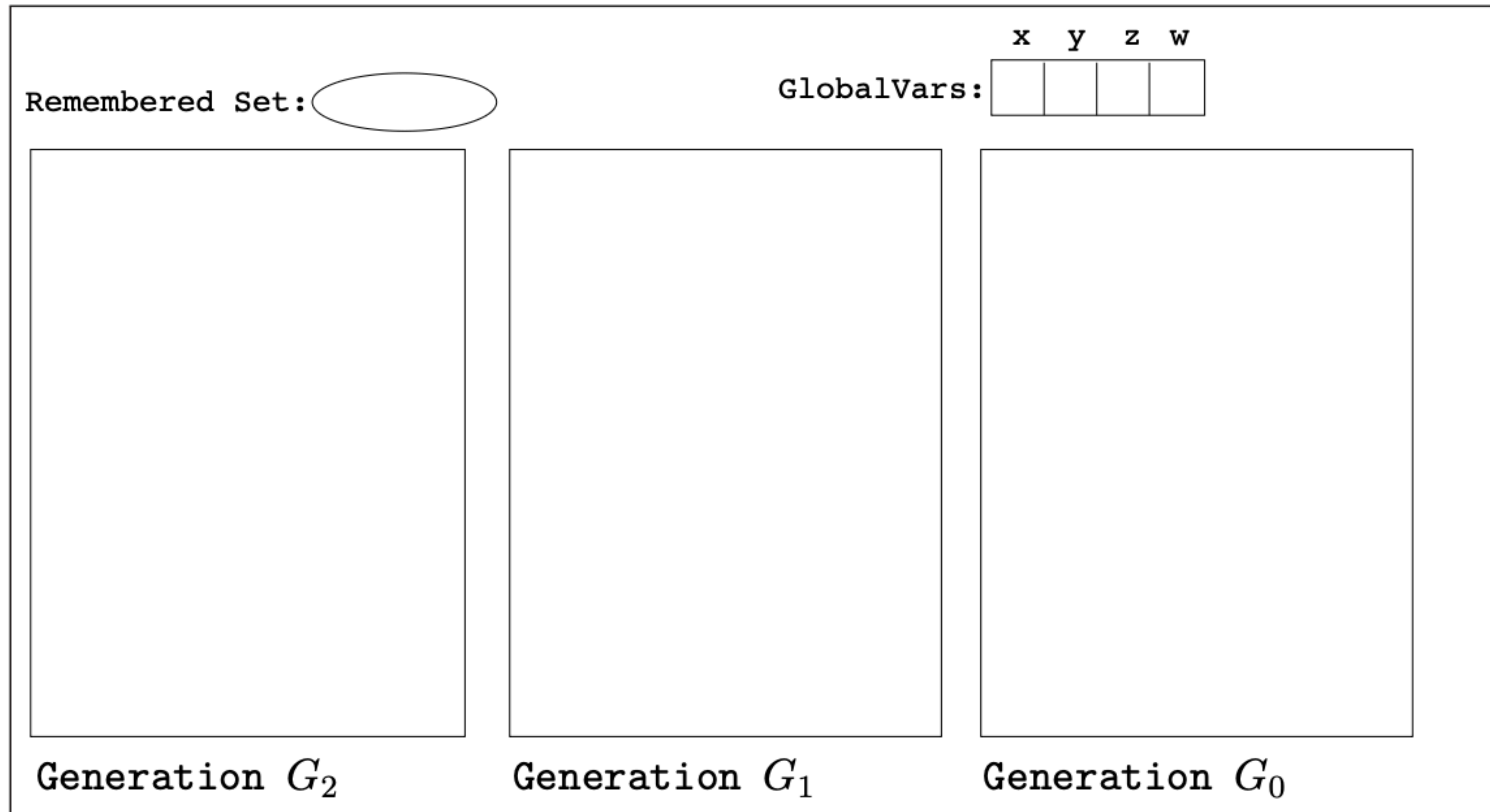

Task 5 (d)

1. Show the state of the heap after time t_2 :



Task 5 (e)

2. Show the state of the heap after time t_9 :



Task 6

- Consider the Java program on the next two pages.

```
class Pair {  
    public java.lang.Object x, y;  
    public char id;  
    public Pair(char id,  
                java.lang.Object x,  
                java.lang.Object y) {  
        this.y = y; this.y = y; this.id = id;  
    }  
}
```



```
public class gc {  
    static Pair p1;  
  
    public static void Q() {  
        java.lang.Integer t3 = new java.lang.Integer(77);  
        Pair p2 = new Pair('b',p1,t3);  
        java.lang.Integer t4 = new java.lang.Integer(88);  
        p1.x = new Pair('c',p1.y,t4);  
    }  
  
    public static void main(String args[]) {  
        java.lang.Integer t1 = new java.lang.Integer(55);  
        java.lang.Integer t2 = new java.lang.Integer(66);  
        p1 = new Pair('a',t1,t2);  
        Q();  
        java.lang.System.gc();  
    }  
}
```

Task 6 (d)

1. Show the state of memory (globals, stack, locals, heap; ignore any object templates) immediately **after** the last statement of `Q ( )` has been executed but **before** `Q ( )` returns.

- Assume that no garbage collection has been performed at this point.
- For each object you should indicate its address in memory.
- Assume that objects are allocated in the order that the `new:s` are executed and that the size of every object is **10 bytes**.

Task 6 (e)

2. Assume that we have just returned from the call to $Q()$.

- At this point we invoke the **mark-and-compact** garbage collector.
- Show the state of memory (globals, stack, locals, heap) immediately **after** the **mark**-part of the garbage collector has executed.
- I.e. you should indicate which objects have been marked.

Task 6 (f)

3. Show the state of memory (globals, stack, locals, heap) right **after** the garbage collector has finished executing.

- For each object you should indicate its new address in memory.



DEFINITIONS AND ALGORITHMS

- Marking phase:

1. Mark all objects *unmarked*.

2. Find all roots, i.e. heap pointers in stack, registers & globals.

3. Mark reachable blocks using a Depth First Search starting at the roots. DFS may run out of stack space; use non-recursive (Deutsch-Schorr-Waite) DFS.

- Sweep phase:

- If an object is not reachable, put it on the free list.

Deutsch-Schorr-Waite Algorithm

- Keep two global variables:
 - `current` points to the current cell,
 - `previous` points to the previously visited cell.
- When a new object is encountered
 1. set the marked bit to 1,
 2. make the object point to the previous node,
 3. update `current` and `previous`.

Compact Algorithm

1. Calculate the forwarding address of each object.
2. Store the forwarding address of object B in
B->forwarding_addr.
3. If p points to object B, replace p with
B->forwarding_addr.
4. Move all object to their forwarding addresses.

1. `scan = next = ADDR(to-space)`
 - `[scan...next]` holds the BFS queue.
 - Objects above `scan` point into `to-space`. Objects between `scan` and `next` point into `from-space`.
 2. Copy objects pointed to by the root pointers to `to-space`.
 3. Update the root pointers to point to `to-space`.
 4. Put each object's new address first in the original.
 5. Repeat (recursively) with all the pointers in the new `to-space`.
 - Update `scan` to point past the last processed node.
 - Update `next` to point past the last copied node.
- Continue while `scan < next`.

COLLBERG.CS.ARIZONA.EDU

LIGERLABS.ORG

SUPPORTED BY
NSF SATC/TTP-1525820
SATC/EDU-2029632

COPYRIGHT © 2024-2026

CHRISTIAN COLLBERG

UNIVERSITY OF ARIZONA